



Universidad Técnica de Babahoyo



AREA DE ESTUDIO
INTELIGENCIA DE NEGOCIOS

ACTIVIDAD # 3.3

CARRERA:
SISTEMAS DE INFORMACIÓN

INGENIERO/A:
ING. FREDY MAXIMILIANO JORDÁN
CORDONES

CURSO:
SEXTO – B

ESTUDIANTE:
CRUZ EMILIA VARGAS VERDEZOTO

LUGAR:
BABAHOYO – ECUADOR

FECHA:
ABRIL 2026 - SEPTIEMBRE 2026

Fase Práctica Autónoma: Desarrolla un script estructurado en Python para cumplir con el siguiente flujo de procesamiento:

Carga de datos: Importa el módulo pandas y carga el archivo `tecno_mega_store.csv`.

```
# Paso 1: Carga de datos
df = pd.read_csv('tecno_mega_store.csv', index_col=0)
```

Feature Engineering Vectorizado: Agrega tres nuevas columnas numéricas al DataFrame utilizando las siguientes fórmulas matemáticas formales:

$$\begin{aligned}\text{Total_Ventas}_i &= \text{Precio_Unitario}_i \times \text{Cantidad}_i \\ \text{Total_Costos}_i &= \text{Costo_Unitario}_i \times \text{Cantidad}_i \\ \text{Margen_Ganancia}_i &= \text{Total_Ventas}_i - \text{Total_Costos}_i\end{aligned}$$

```
# Paso 2: Feature Engineering vectorizado
df['Total_Ventas'] = df['Precio_Unitario'] * df['Cantidad']
df['Total_Costos'] = df['Costo_Unitario'] * df['Cantidad']
df['Margen_Ganancia'] = df['Total_Ventas'] - df['Total_Costos']
```

Filtrado Multicondicional Complejo: Construye máscaras booleanas para aislar los siguientes dos segmentos estratégicos:

Segmento alerta_rentabilidad: Registros con un volumen operativo alto pero retribución deficiente, definidos matemáticamente bajo la intersección lógica: $(\text{Cantidad} \geq 10) \text{ AND } (\text{Margen_Ganancia} < 100)$.

Segmento clientes_vip: Ventas de gran escala pertenecientes a las líneas de mayor valor tecnológico: $(\text{Categoria} \in \{\text{'Computadoras'}, \text{'Televisores'}\}) \text{ AND } (\text{Total_Ventas} > 5000)$.

```
# Paso 3: Filtrado multicondicional complejo
alerta_rentabilidad = df[(df['Cantidad'] >= 10) & (df['Margen_Ganancia'] < 100)]

clientes_vip = df[
    (df['Categoria'].isin(['Computadoras', 'Televisores'])) &
```

```
(df['Total_Ventas'] > 5000)
]
```

Verificación Dimensional: Imprime la propiedad .shape de ambos sub-datasets para registrar su volumen en el reporte final.

```
# Paso 4: Verificacion dimensional
print('Shape alerta_rentabilidad:', alerta_rentabilidad.shape)
print('Shape clientes_vip:', clientes_vip.shape)
print(alerta_rentabilidad[['Producto', 'Cantidad', 'Margen_Ganancia']])
print(clientes_vip[['Producto', 'Categoria', 'Total_Ventas']])
```

Código completo:

```
import pandas as pd

# Paso 1: Carga de datos
df = pd.read_csv('tecno_mega_store.csv', index_col=0)

# Paso 2: Feature Engineering vectorizado
df['Total_Ventas'] = df['Precio_Unitario'] * df['Cantidad']
df['Total_Costos'] = df['Costo_Unitario'] * df['Cantidad']
df['Margen_Ganancia'] = df['Total_Ventas'] - df['Total_Costos']

# Paso 3: Filtrado multicondicional complejo
alerta_rentabilidad = df[(df['Cantidad'] >= 10) & (df['Margen_Ganancia'] < 100)]

clientes_vip = df[
    (df['Categoria'].isin(['Computadoras', 'Televisores'])) &
    (df['Total_Ventas'] > 5000)
]

# Paso 4: Verificacion dimensional
print('Shape alerta_rentabilidad:', alerta_rentabilidad.shape)
print('Shape clientes_vip:', clientes_vip.shape)
print(alerta_rentabilidad[['Producto', 'Cantidad', 'Margen_Ganancia']])
print(clientes_vip[['Producto', 'Categoria', 'Total_Ventas']])
```

Captura:

```
[13]: import pandas as pd

# Paso 1: Carga de datos
df = pd.read_csv('tecno_mega_store.csv', index_col=0)

# Paso 2: Feature Engineering vectorizado
df['Total_Ventas'] = df['Precio_Unitario'] * df['Cantidad']
df['Total_Costos'] = df['Costo_Unitario'] * df['Cantidad']
df['Margen_Ganancia'] = df['Total_Ventas'] - df['Total_Costos']

# Paso 3: Filtrado multicondicional complejo
alerta_rentabilidad = df[(df['Cantidad'] >= 10) & (df['Margen_Ganancia'] < 100)]

clientes_vip = df[
    (df['Categoria'].isin(['Computadoras', 'Televisores'])) &
    (df['Total_Ventas'] > 5000)
]

# Paso 4: Verificacion dimensional
print('Shape alerta_rentabilidad:', alerta_rentabilidad.shape)
print('Shape clientes_vip:', clientes_vip.shape)
print(alerta_rentabilidad[['Producto', 'Cantidad', 'Margen_Ganancia']])
print(clientes_vip[['Producto', 'Categoria', 'Total_Ventas']])

... Shape alerta_rentabilidad: (4, 13)
Shape clientes_vip: (7, 13)
      Producto  Cantidad  Margen_Ganancia
1  Memoria USB 64GB SanDisk      16         51.84
22 Cable HDMI 2.0 (2m)      12         25.20
39 Webcam HD 1080p      14         98.14
89 Cargador USB-C 65W      14         96.46
      Producto  Categoria  Total_Ventas
3  PC Escritorio Asus M700  Computadoras    12300.0
16 Laptop Lenovo IdeaPad 3  Computadoras     5592.0
42 Laptop HP 250 G9  Computadoras    17976.0
46 Laptop HP 250 G9  Computadoras    14980.0
62 Laptop Dell Inspiron 15  Computadoras     5803.0
67 TV LG 50 pulg 4K UHD  Televisores    10059.0
79 Laptop HP 250 G9  Computadoras     9737.0
```

Redacta un breve análisis (máximo 4 líneas) respondiendo la siguiente interrogante: ¿Qué recomendaciones operativas o comerciales le sugerirías a la gerencia respecto a los productos detectados en el segmento "alerta_rentabilidad"? Justifica tu respuesta considerando el esfuerzo de inventario frente a la ganancia real obtenida.

El segmento alerta_rentabilidad reúne accesorios de bajo precio unitario (memorias, cables, webcams, cargadores) que se venden en volumen pero dejan menos de \$100 de ganancia. Sugieren revisar el margen de compra/venta de esa línea de accesorios y evitar sobre-invertir esfuerzo logístico en productos de bajo retorno.

Actividad 2 · El Campeonato de los Datos (Agrupamiento y Rankings de Mercado)

Fase Práctica Autónoma: Desarrolla el procesamiento correspondiente en tu script de Python:

Integración de Datos: Retoma el DataFrame original y calcula la columna de ingresos Total_Ventas procesada en la Actividad 1.

Resumen Multivariable: Agrupa los registros basándote en la columna Categoría. Aplica el método de agregación .agg() para calcular en una sola operación:

La suma acumulada de la métrica Total_Ventas.

El promedio de unidades transaccionadas (columna Cantidad).

La cantidad de productos únicos ofertados (usando la función de conteo de cardinalidad 'nunique' sobre la variable Producto).

```
# Paso 1: Resumen multivariable con .agg()
resumen = df.groupby('Categoría').agg({
    'Total_Ventas': 'sum',
    'Cantidad': 'mean',
    'Producto': 'nunique'
})
```

Aplanamiento e Indexación: Ejecuta la técnica avanzada explicada en la guía para colapsar los nombres de columna multinivel resultantes. Renombra las columnas con etiquetas claras (ej. Ventas_Totales, Promedio_Unidades, Modelos_Unicos).

```
# Paso 2: Aplanamiento y renombrado
resumen.columns = ['Ventas_Totales', 'Promedio_Unidades',
                  'Modelos_Unicos']
resumen = resumen.reset_index()
```

Jerarquías del Rendimiento: Desarrolla los siguientes ordenamientos:

Extrae el Top 3 de Categorías Líderes (ordenado de forma descendente en ventas).

Extrae el Top 3 de Regiones Críticas (agrupa por la columna Region, calcula la suma de ventas y ordena de manera ascendente para evidenciar las zonas geográficas con menor dinamismo comercial).

```
# Paso 3: Rankings
top3_categorias = resumen.sort_values('Ventas_Totales',
ascending=False).head(3)

top3_regiones_criticas = (
    df.groupby('Region')['Total_Ventas'].sum()
      .sort_values(ascending=True)
      .head(3)
)
```

Síntesis e Interpretación de Negocio (Meta-cognición):

Analiza la tabla final obtenida en tu script y responde: ¿La categoría que acumuló el mayor volumen de "Ventas_Totales" registra también la mayor cantidad de "Promedio_Unidades" transaccionadas? Explica la razón de este comportamiento considerando la diferencia de precios entre productos (ej. un accesorio barato frente a una computadora).

Computadoras lidera en ventas totales, pero no tiene el mayor Promedio_Unidades (ese lo tiene Perifericos, con 17), esto pasa porque el ingreso en dólares depende del precio unitario y no solo de las unidades vendidas pocas computadoras de alto valor generan más ingreso que muchas unidades de accesorios baratos.

Código Completo:

```

# Paso 1: Resumen multivariable con .agg()
resumen = df.groupby('Categoria').agg({
    'Total_Ventas': 'sum',
    'Cantidad': 'mean',
    'Producto': 'nunique'
})

# Paso 2: Aplanamiento y renombrado
resumen.columns = ['Ventas_Totales', 'Promedio_Unidades',
'Modelos_Unicos']
resumen = resumen.reset_index()

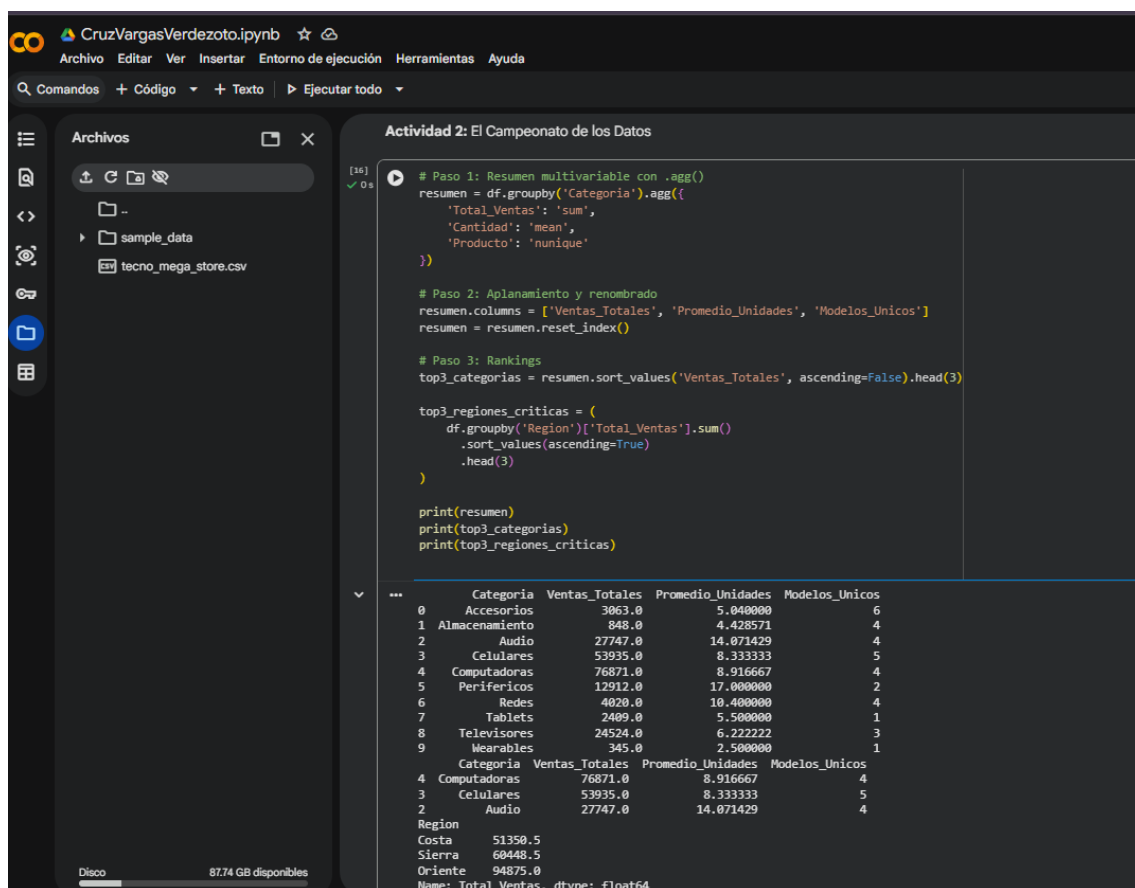
# Paso 3: Rankings
top3_categorias = resumen.sort_values('Ventas_Totales',
ascending=False).head(3)

top3_regiones_criticas = (
    df.groupby('Region')['Total_Ventas'].sum()
    .sort_values(ascending=True)
    .head(3)
)

print(resumen)
print(top3_categorias)
print(top3_regiones_criticas)

```

Captura:



Actividad 2: El Campeonato de los Datos

```

# Paso 1: Resumen multivariable con .agg()
resumen = df.groupby('Categoria').agg({
    'Total_Ventas': 'sum',
    'Cantidad': 'mean',
    'Producto': 'nunique'
})

# Paso 2: Aplanamiento y renombrado
resumen.columns = ['Ventas_Totales', 'Promedio_Unidades', 'Modelos_Unicos']
resumen = resumen.reset_index()

# Paso 3: Rankings
top3_categorias = resumen.sort_values('Ventas_Totales', ascending=False).head(3)

top3_regiones_criticas = (
    df.groupby('Region')['Total_Ventas'].sum()
    .sort_values(ascending=True)
    .head(3)
)

print(resumen)
print(top3_categorias)
print(top3_regiones_criticas)

```

	Categoria	Ventas_Totales	Promedio_Unidades	Modelos_Unicos
0	Accesorios	3063.0	5.040000	6
1	Almacenamiento	848.0	4.428571	4
2	Audio	27747.0	14.071429	4
3	Celulares	53935.0	8.333333	5
4	Computadoras	76871.0	8.916667	4
5	Periféricos	12912.0	17.000000	2
6	Redes	4020.0	10.400000	4
7	Tablets	2409.0	5.500000	1
8	Televisores	24524.0	6.222222	3
9	Wearables	345.0	2.500000	1

	Categoria	Ventas_Totales	Promedio_Unidades	Modelos_Unicos
4	Computadoras	76871.0	8.916667	4
3	Celulares	53935.0	8.333333	5
2	Audio	27747.0	14.071429	4

	Region	Total_Ventas
0	Costa	51350.5
1	Sierra	60448.5
2	Oriente	94875.0

Name: Total Ventas, dtype: float64

Actividad 3 · El Detective del Tiempo y el Espacio (Dinámicas Temporales y Matriciales)

Fase Práctica Autónoma: Incorpora al archivo de comandos el procesamiento final de tu flujo:

Tipado y Cast Temporal: Convierte la columna Fecha de formato string a tipos nativos datetime64 mediante la instrucción `pd.to_datetime()`. Asígnala como índice estructural del DataFrame.

```
# Paso 1: Tipado y cast temporal
df['Fecha'] = pd.to_datetime(df['Fecha'])
df = df.set_index('Fecha')
```

Muestreo Periódico (Resampling): Agrupa cronológicamente tus transacciones con frecuencia de **Fin de Mes ('ME')** y calcula la suma totalizada del volumen de productos vendidos (Cantidad). Imprime el vector resultante.

```
# Paso 2: Resampling mensual (fin de mes)
ventas_mensuales = df.resample('ME')['Cantidad'].sum()
```

Extracción de Componentes: Libera el índice del DataFrame y utiliza el accesor temporal de Pandas (`.dt.month_name()`) para generar una nueva variable de control categórica denominada `Mes_Nombre`.

```
# Paso 3: Liberar índice y extraer componente de fecha
df = df.reset_index()
df['Mes_Nombre'] = df['Fecha'].dt.month_name()
```

Cruce Bidimensional de Mercado: Genera una matriz de información ejecutiva aplicando `.pivot_table()` utilizando las siguientes dimensiones:

En las **filas** (index): Ubica la variable territorial `Region`.

En las **columnas** (columns): Coloca la segmentación `Tipo_Cliente`.

En el ****centro (valores)**** (values): Procesa la agregación de la columna `Total_Ventas` mediante la función `'sum'`.

Para agregar totales generales a nivel de fila y columna, activa la propiedad `margins=True`.

```
# Paso 4: Tabla dinamica con margenes
```

```
matriz = df.pivot_table(  
    index='Region',  
    columns='Tipo_Cliente',  
    values='Total_Ventas',  
    aggfunc='sum',  
    margins=True  
)
```

```
print(ventas_mensuales)
```

```
print(matriz.round(2))
```

Código Completo:

```
# Paso 1: Tipado y cast temporal
```

```
df['Fecha'] = pd.to_datetime(df['Fecha'])
```

```
df = df.set_index('Fecha')
```

```
# Paso 2: Resampling mensual (fin de mes)
```

```
ventas_mensuales = df.resample('ME')['Cantidad'].sum()
```

```
# Paso 3: Liberar indice y extraer componente de fecha
```

```
df = df.reset_index()
```

```
df['Mes_Nombre'] = df['Fecha'].dt.month_name()
```

```
# Paso 4: Tabla dinamica con margenes
```

```
matriz = df.pivot_table(  
    index='Region',  
    columns='Tipo_Cliente',  
    values='Total_Ventas',  
    aggfunc='sum',  
    margins=True  
)
```

```
print(ventas_mensuales)
```

```
print(matriz.round(2))
```

Captura:

CruzVargasVerdezoto.ipynb

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda

Comandos + Código + Texto Ejecutar todo

Archivos

-
- sample_data
- tecno_mega_store.csv

Actividad 3: El Detective del Tiempo y el Espacio

```

# Paso 1: Tipado y cast temporal
df['Fecha'] = pd.to_datetime(df['Fecha'])
df = df.set_index('Fecha')

# Paso 2: Resampling mensual (fin de mes)
ventas_mensuales = df.resample('ME')['Cantidad'].sum()

# Paso 3: Liberar indice y extraer componente de fecha
df = df.reset_index()
df['Mes_Nombre'] = df['Fecha'].dt.month_name()

# Paso 4: Tabla dinamica con margenes
matriz = df.pivot_table(
    index='Region',
    columns='Tipo_Cliente',
    values='Total_Ventas',
    aggfunc='sum',
    margins=True
)

print(ventas_mensuales)
print(matriz.round(2))

```

```

--- Fecha
2024-01-31    50
2024-02-29    49
2024-03-31    24
2024-04-30   100
2024-05-31    83
2024-06-30    62
2024-07-31    96
2024-08-31    20
2024-09-30    50
2024-10-31    29
2024-11-30    57
2024-12-31    51
2025-01-31    11
2025-02-28    28
2025-03-31    53
2025-04-30    49
2025-05-31    13
2025-06-30     5
Freq: ME, Name: Cantidad, dtype: int64

```

Tipo_Cliente	Consumidor	Final	Corporativo	Mayorista	Minorista	All
Region						
Costa		5690.5	18668.0	12513.0	14479.0	51350.5
Oriente		11073.0	41567.0	38570.0	3665.0	94875.0
Sierra		6246.0	16509.0	30248.0	7445.5	60448.5
All		23009.5	76744.0	81331.0	25589.5	206674.0

Síntesis e Interpretación de Negocio (Meta-cognición):

Inspecciona la matriz obtenida en tu salida de consola y documenta: ¿Cuál es la combinación regional y tipo de cliente que genera la mayor concentración de ingresos económicos para Tecno Mega Store? ¿Cuál representa el menor aporte comercial? Propón una iniciativa estratégica o promocional dirigida específicamente al sector que menos ingresos te aporta basándote en los perfiles de compra de dicha región.

La combinación con mayor ingreso es Oriente + Corporativo (41,567), seguida de Oriente + Mayorista (38,570). La de menor aporte es Oriente + Minorista (3,665). Como iniciativa se podría proponer una promoción dirigida a pequeños comercios minoristas de la región Oriente, aprovechando la logística ya existente hacia esa zona con los canales Corporativo y Mayorista.

Link de Google colap.

https://colab.research.google.com/drive/1hn3rht7aa4YjC9Wfo6dUVI0-O_NRIUO8?usp=sharing